Массивы в numpy

Шпаргалка



Функции из урока «Массивы в numpy» и ещё немного — по задачам: что нужно сделать, чем, как это выглядит. Под каждой строкой пример; после стрелки — его значение, снятое запуском. Все примеры — на этих массивах:

```
import numpy as np
a = np.array([3, 1, 2])
T = np.array([[1, 2, 3], [4, 5, 6]])
x = np.array([2.0, np.nan, 4.0])
g = np.array([0, 2, 0, 1])
v = np.array([1.0, 2.0, 3.0, 4.0])
w = np.array(["b", "a", "b"])
A = np.array([[2.0, 1.0], [1.0, 3.0]])
rng = np.random.default_rng(42)
swv = np.lib.stride_tricks.sliding_window_view
```

Создать

`np.array` — массив из списка; одно дробное — все дробные

`np.array([1, 2.5])` → `[1.  2.5]`

`np.arange` — от, до — не включая, шаг

`np.arange(2, 11, 3)` → `[2 5 8]`

`np.linspace` — n точек от и до, оба конца входят

`np.linspace(0, 1, 5)` → `[0.   0.25 0.5  0.75 1.  ]`

`np.full` — массив из одного числа; ещё `np.zeros`, `np.ones`

`np.full(3, 7)` → `[7 7 7]`

`np.zeros_like` — нули той же формы и типа

`np.zeros_like(T).shape` → `(2, 3)`

`np.eye` — единичная матрица

`np.eye(3).sum()` → `3.0`

`rng.integers` — случайные целые, конец не входит

```
rng.integers(1, 7, size=5) → [1 5 4 3 3]
```

`rng.normal` — нормальные: среднее, разброс

```
rng.normal(0, 1, size=2).round(2) → [ 0.94 -1.95]
```

`rng.choice` — выбор из массива; `replace=False` — без повторов

```
rng.choice(a, size=2, replace=False) → [1 2]
```

`rng.permutation` — случайная перестановка

```
rng.permutation(4) → [2 3 0 1]
```

Узнать форму и тип

`a.shape` — форма: длина по каждой оси

```
T.shape → (2, 3)
```

`a.ndim` — число осей

```
T.ndim → 2
```

`a.size` — всего элементов

```
T.size → 6
```

`a.dtype` — тип элементов

```
T.dtype → int64
```

`a.astype` — копия другого типа; дробное в целое — к нулю

```
np.array([2.7, -2.7]).astype(int) → [ 2 -2]
```

Взять часть

`a[i]` — элемент по номеру, с нуля; `a[-1]` — последний

```
a[-1] → 2
```

`a[i:j:k]` — срез: от, до — не включая, шаг; это вид, не копия

```
a[::-1] → [2 1 3]
```

`T[i, j]` — элемент таблицы: номер по каждой оси

```
T[1, 2] → 6
```

`T[:, j]` — столбец: «все строки, столбец j»

```
T[:, 1] → [2 5]
```

`a[[i, j]]` — по списку номеров, с повторами; копия

```
a[[2, 0, 0]] → [2 3 3]
```

`a[mask]` — по маске из `True` и `False`; копия

```
a[a > 1] → [3 2]
```

`np.where` — поэлементно «если, то, иначе»

```
np.where(a > 1, a, 0) → [3 0 2]
```

`np.flatnonzero` — номера, где маска верна

```
np.flatnonzero(a > 1) → [0 2]
```

`a.copy` — копия, которую можно менять

```
b = a.copy()
```

`np.shares_memory` — смотрят ли два массива в одни данные

```
np.shares_memory(a, a[1:]) → True
```

Поменять форму

`a.reshape` — те же числа в другой форме; -1 — «сколько выйдет»

```
np.arange(6).reshape(-1, 2).shape → (3, 2)
```

`a.ravel` — всё в одну строку

```
T.ravel() → [1 2 3 4 5 6]
```

`a.T` — транспонирование: строки становятся столбцами

```
T.T.shape → (3, 2)
```

`a[:, None]` — новая ось длины 1: ряд в столбец

```
a[:, None].shape → (3, 1)
```

`np.stack` — сложить массивы по новой оси

```
np.stack([a, a]).shape → (2, 3)
```

`np.concatenate` — приставить по существующей оси

```
np.concatenate([a, [9]]) → [3 1 2 9]
```

`np.squeeze` — убрать оси длины 1

```
np.squeeze(a[:, None]).shape → (3,)
```

Посчитать

`a.sum` — сумма; с `axis` — вдоль оси, и её не станет

```
T.sum(axis=0) → [5 7 9]
```

`a.mean` — среднее; так же `a.min()`, `a.max()`, `a.std()`

```
T.mean(axis=1) → [2. 5.]
```

`keepdims=True` — оставить свёрнутую ось длиной 1

```
T.sum(axis=1, keepdims=True).shape → (2, 1)
```

`axis=(0, 1)` — свернуть несколько осей сразу

```
T.max(axis=(0, 1)) → 6
```

`a.argmax` — номер наибольшего; `a.argmin()` — наименьшего

```
T.argmax(axis=1) → [2 2]
```

`np.median` — медиана

```
np.median([1, 5, 2, 8]) → 3.5
```

`np.percentile` — процентиль: ниже него заданная доля

```
np.percentile([1, 2, 3, 4], 50) → 2.5
```

`a.any` — есть ли хоть один `True`; `a.all()` — все ли

```
(a > 2).any() → True
```

`np.count_nonzero` — сколько ненулевых и `True`

```
np.count_nonzero(a > 1) → 2
```

`np.sqrt` — поэлементно; ещё `np.abs`, `np.exp`, `np.log`

```
np.sqrt([4, 9]) → [2. 3.]
```

`np.maximum` — наибольшее поэлементно из двух

```
np.maximum(a, 2) → [3 2 2]
```

`np.clip` — обрезать до границ

```
np.clip([0, 5, 9], 1, 8) → [1 5 8]
```

`np.round` — к ближайшему, половину — к чётному

```
np.round([0.5, 1.5, 2.5]) → [0. 2. 2.]
```

`np.floor` — вниз; `np.ceil` — вверх

```
np.floor([-2.7, 2.7]) → [-3.  2.]
```

Пропуски и сравнение дробных

`np.nan` — пропуск; бывает только в дробном массиве

```
np.nan == np.nan → False
```

`np.isnan` — маска пропусков

```
np.isnan(x) → [False  True False]
```

`np.nanmean` — счёт мимо пропусков; ещё `np.nansum`, `np.nanmax`

```
np.nanmean(x) → 3.0
```

`np.nan_to_num` — заменить пропуски числом

```
np.nan_to_num(x, nan=0) → [2. 0. 4.]
```

`np.isclose` — равны с точностью до округлений

```
np.isclose(0.1 + 0.2, 0.3) → True
```

`np.allclose` — то же для массивов целиком

```
np.allclose([1, 2], [1, 2 + 1e-12]) → True
```

Сортировать

`np.sort` — копия по возрастанию; `a.sort()` — на месте

```
np.sort(a) → [1 2 3]
```

`np.argsort` — номера элементов по возрастанию

```
np.argsort(a) → [1 2 0]
```

`kind="stable"` — равные — в исходном порядке

```
np.argsort([2, 1, 2], kind="stable") → [1 0 2]
```

`-a` — по убыванию — сортировать `-a`

```
np.argsort(-a, kind="stable") → [0 2 1]
```

`np.argpartition` — номера k наименьших, без порядка между ними

```
np.argpartition(a, 1)[:2] → [1 2]
```

`np.searchsorted` — куда вставить, чтобы порядок сохранился

```
np.searchsorted([1, 3, 5], 4) → 2
```

Группы и счёт

`np.unique` — различные значения по возрастанию

```
np.unique([3, 1, 3]) → [1 3]
```

`return_counts=True` — и сколько раз каждое

```
np.unique(g, return_counts=True)[1] → [2 1 1]
```

`return_inverse=True` — номер группы у каждого элемента

```
np.unique(w, return_inverse=True)[1] → [1 0 1]
```

`np.bincount` — сколько раз встретился каждый номер $0, 1, 2, \ldots$

```
np.bincount(g) → [2 1 1]
```

`weights=` — сумма весов по номерам

```
np.bincount(g, weights=v) → [4. 4. 2.]
```

`minlength=` — добавить номера, которых нет, со счётом 0

```
np.bincount(g, minlength=5) → [2 1 1 0 0]
```

`np.isin` — маска «равно одному из»

```
np.isin(a, [1, 2]) → [False  True  True]
```

`np.histogram` — счёт по интервалам

```
np.histogram([1, 2, 2, 5], bins=2)[0] → [3 1]
```

Накопления и окна

`np.cumsum` — накопленные суммы

```
np.cumsum(a) → [3 4 6]
```

`np.cumprod` — накопленные произведения

```
np.cumprod(a) → [3 3 6]
```

`np.diff` — разности соседних

```
np.diff(a) → [-2  1]
```

`sliding_window_view` — все окна строками, без копии; здесь `swv`

```
swv(np.arange(5), 3).sum(axis=1) → [3 6 9]
```

`np.convolve` — скользящая сумма с весами

```
np.convolve(a, [1, 1], "valid") → [4 3]
```

Векторы и матрицы

`@` — произведение матриц, матрицы на вектор, векторов

```
T @ np.array([1, 0, 1]) → [ 4 10]
```

`np.dot` — скалярное произведение

```
np.dot([1, 2], [3, 4]) → 11
```

`np.outer` — таблица всех произведений

```
np.outer([1, 2], [1, 2, 3]).shape → (2, 3)
```

`np.linalg.norm` — длина; с `axis=1` — длины строк

```
np.linalg.norm([3, 4]) → 5.0
```

`np.linalg.solve` — решить $Ax = b$

```
np.linalg.solve(A, [3, 5]) → [0.8 1.4]
```

`np.linalg.inv` — обратная матрица; для $Ax = b$ лучше `solve`

```
np.linalg.inv(A) @ [3, 5] → [0.8 1.4]
```

`np.linalg.det` — определитель

```
round(np.linalg.det(A), 6) → 5.0
```

Файлы

`np.save` — один массив в файл `.npy`

```
np.save("a.npy", a)
```

`np.load` — прочитать `.npy` или `.npz`

```
np.load("a.npy") → [3 1 2]
```

`np.savez` — несколько массивов под именами, `.npz`

```
np.savez("d.npz", a=a, T=T)
```

`.files` — имена массивов в `.npz`

```
np.load("d.npz").files → ['a', 'T']
```

`np.loadtxt` — числа из текстового файла

```
np.loadtxt("chisla.txt").shape → (2, 2)
```

`np.savetxt` — записать массив текстом

```
np.savetxt("t.txt", T, fmt="%d")
```